

PRACTICAL NO. 7

AIM: A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint) The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below)

Process	Allocation Printer	Allocation ROM	Allocation Hard Disk	Allocation RAM	Max Printer	Max ROM	Max Hard Disk	Max RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoin t	1	3	5	4	2	3	5	6

Available Resources			
Printer	ROM	Hard Disk	RAM
1	6	2	0

Write a program to:

- Calculate current need of resources by each process Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:

MINGW64:/c:/Users/airat/OneDrive/Desktop

GNU nano 8.7

banker.c

```
#include <stdio.h>

#define P 5 // Processes
#define R 4 // Resources

int main() {

    int i, j, k;
    int allocation[P][R] = {
        {0,0,1,4}, // Google Drive
        {0,6,3,2}, // Firefox
        {0,0,1,2}, // Word
        {1,0,0,0}, // Excel
        {1,3,5,4} // PowerPoint
    };

    int max[P][R] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[R] = {1,6,2,0};

    int need[P][R];
    int finish[P] = {0};
    int safeSeq[P];
    int work[R];

    printf("\n===== ALLOCATION MATRIX =====\n");
    for(i=0;i<P;i++){
        for(j=0;j<R;j++){
            printf("%d ", allocation[i][j]);
        }
        printf("\n");
    }

    printf("\n===== MAX MATRIX =====\n");
    for(i=0;i<P;i++){
        for(j=0;j<R;j++){
            printf("%d ", max[i][j]);
        }
        printf("\n");
    }
}
```

GNU nano 8.7

```
    printf("\n");
}

printf("\n==== NEED MATRIX ==== \n");
for(i=0;i<P;i++){
    for(j=0;j<R;j++){
        need[i][j] = max[i][j] - allocation[i][j];
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

printf("\nAvailable Resources:\n");
for(i=0;i<R;i++)
    printf("%d ", available[i]);

printf("\n");

for(i=0;i<R;i++)
    work[i] = available[i];

int count = 0;

while(count < P) {
    int found = 0;

    for(i=0;i<P;i++) {
        if(finish[i] == 0) {

            int flag = 0;
            for(j=0;j<R;j++) {
                if(need[i][j] > work[j]) {
                    flag = 1;
                    break;
                }
            }

            if(flag == 0) {
                for(k=0;k<R;k++)
                    work[k] += allocation[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
}
```

```
int count = 0;

while(count < P) {
    int found = 0;

    for(i=0;i<P;i++) {
        if(finish[i] == 0) {

            int flag = 0;
            for(j=0;j<R;j++) {
                if(need[i][j] > work[j]) {
                    flag = 1;
                    break;
                }
            }

            if(flag == 0) {
                for(k=0;k<R;k++)
                    work[k] += allocation[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0) {
        printf("\nSystem is NOT in Safe State.\n");
        return 0;
    }
}

printf("\nSystem is in SAFE State.\n");
printf("Safe Sequence: ");

for(i=0;i<P;i++)
    printf("%d ", safeSeq[i]);

printf("\n");

return 0;
}
```

OUTPUT :

```
MINGW64:/c/Users/airat/OneDrive/Desktop
Faiz@PUCKC3F09N9DT MINGW64 ~/OneDrive/Desktop (master)
$ nano banker.c
Faiz@PUCKC3F09N9DT MINGW64 ~/OneDrive/Desktop (master)
$ gcc banker.c -o banker
Faiz@PUCKC3F09N9DT MINGW64 ~/OneDrive/Desktop (master)
$ ./banker

===== ALLOCATION MATRIX =====
0 0 1 4
0 6 3 2
0 0 1 2
1 0 0 0
1 3 5 4

===== MAX MATRIX =====
0 6 5 6
0 6 5 2
0 0 1 2
1 7 5 0
2 3 5 6

===== NEED MATRIX =====
0 6 4 2
0 0 2 0
0 0 0 0
0 7 5 0
1 0 0 2

Available Resources:
1 6 2 0

System is in SAFE State.
Safe Sequence: P1 P2 P3 P4 P0
Faiz@PUCKC3F09N9DT MINGW64 ~/OneDrive/Desktop (master)
$ |
```